



Lesson 6.6 — RSC vs HTML vs JavaScript

This is a **very important interview topic**, because you've learned Server Components, Client Components, SSR, streaming, PPR, and caching — but now we need to see **what actually travels over the network**.

Let's start from React + Parcel, because you already know that model.

1. React + Parcel first

In a traditional React app:

```
graph TD; Browser1[Browser] --> GET[GET /]; GET --> Server[Server]; Server --> index[index.html]; index --> Browser2[Browser]; Browser2 --> download[download JS bundle]; download --> React[React executes]; React --> UI[React creates UI]; UI --> DOM[DOM];
```

Your `App.tsx` ultimately gets bundled into JavaScript.

For example:

```
function App() {  
  return <h1>Hello</h1>;  
}
```

becomes part of a JS bundle.

The browser downloads something conceptually like:

index.html



main.js



React code



App code



dependencies

Then JavaScript runs:

main.js



React



App()



Virtual DOM



DOM

2. Now Next.js App Router

This is fundamentally different.

Suppose:

```
export default function Page() {  
  return <h1>Hello</h1>;  
}
```

This is a **Server Component** by default.

The browser does **not** need the `Page` component's JavaScript just to render that UI.

Instead:

Server



Server Component rendering



HTML + RSC payload

↓
Browser

That's a huge architectural difference.

3. What is RSC?

RSC = **React Server Components**.

Important:

RSC is an architecture/protocol for representing the result of Server Components.

Don't think:

RSC = HTML



They're different.

Think:

Server Component
↓
React renders it
↓
RSC payload
↓
React on browser understands it



The RSC payload contains information representing the Server Component tree/result.

4. Why don't we just send HTML?

Great question.

Suppose:

```
export default function Page() {  
  return <Product />;  
}
```

Server could send:

```
<h1>iPhone</h1>
```

But Next.js also needs to maintain the **React component tree** and coordinate it with Client Components.

For example:

```
<Product />
```

```
<AddToCartButton />
```

Maybe:

```
Product
  ↓
Server Component

AddToCartButton
  ↓
Client Component
```

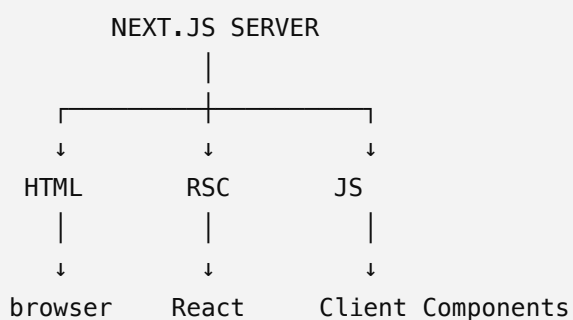


The browser needs information about where the Client Component belongs and what props it receives.

That's where the RSC payload becomes important.

5. Three things to keep separate

This is the mental model I want you to memorize:



They have different purposes.

HTML

Used for:

initial document/UI



RSC payload

Used by React to understand/update the Server Component tree.

JavaScript

Used for:

Client Components
interactivity
event handlers
browser-side logic



6. Example

Let's make this concrete.

```
export default async function ProductPage() {  
  const product = await getProduct();  
  
  return (  
    <main>  
      <h1>{product.name}</h1>  
  
      <AddToCartButton productId={product.id} />  
    </main>  
  );  
}
```

Here:

```
ProductPage  
  ↓  
Server Component  
  
AddToCartButton  
  ↓  
Client Component
```



because maybe:

```
"use client";

export function AddToCartButton({ productId }) {
  return (
    <button onClick={() => addToCart(productId)}>
      Add to cart
    </button>
  );
}
```

7. What happens on the initial request?

Browser:

GET /products/123



Next.js:

Server
↓
execute ProductPage
↓
fetch product
↓
render Server Components



Then Next.js can produce the initial response involving:

HTML
+
RSC information



The browser receives HTML and can display the page.

So the user can see:

iPhone
₹79,999
[Add to cart]



8. But wait...

You might say:

"If AddToCartButton is a Client Component, doesn't its JavaScript need to reach the browser?"

YES. 🎯

That's the important distinction.

The **Server Component's JavaScript doesn't need to be shipped to the browser for execution**, but the Client Component's JavaScript does.

So:

```
Server Component
  ↓
server executes it
  ↓
browser doesn't need its component JS
```



while:

```
Client Component
  ↓
browser needs its JS
  ↓
hydration/interactivity
```



9. Hydration enters here

Suppose the browser receives:

```
<button>Add to cart</button>
```

Initially, it may look correct.

But the browser needs JavaScript to make:

```
onClick={() => addToCart(...)}
```

actually work.

So:

```
HTML
↓
visible UI

Client JS
↓
React attaches behavior
↓
interactive UI
```



That's hydration.

10. This is why "use client" matters

When you write:

```
"use client";

export function AddToCartButton() {
  return (
    <button onClick={handleClick}>
      Add
    </button>
  );
}
```

you're saying:

This component belongs to the client side of the component tree and may need browser JavaScript/interactivity.

That does **not** mean:

"The entire page becomes client-side."

Only that component and the relevant client subtree are treated as client components.

11. Now navigation gets interesting

Suppose you're already on:

```
/products/123
```



and click:

```
<Link href="/products/456">  
  Product 456  
</Link>
```



Next.js can perform a client-side navigation.

Instead of doing a traditional:

```
Browser  
↓  
GET entire document  
↓  
replace whole page
```



Next.js can request the relevant **RSC payload**.

Conceptually:

```
Browser  
↓  
click Link  
↓  
Next.js client router  
↓  
request Server Component result  
↓  
RSC payload  
↓  
React reconciles tree  
↓  
UI updates
```



This is one reason App Router navigation feels different from a normal full page reload.

12. Now connect this to your Router Cache

You already learned Router Cache.

So:

```
Server
  ↓
RSC payload
  ↓
Browser
  ↓
Router Cache
```



The client router can retain previously received RSC data for navigation purposes.

So when navigating:

```
/products/123
  ↓
/products/456
```



the browser doesn't necessarily need a completely new HTML document.

It can receive the relevant Server Component payload and update the existing React tree.

13. Now connect PPR

Remember our PPR page:

```
Header
Product
Recommendations
Cart
```



Imagine:

Header	→ pre-renderable
Product	→ cached
Recommendations	→ cached
Cart	→ dynamic



Then the response can conceptually progress like:

```
Server
  ↓
prepared content
  ↓
HTML/RSC
  ↓
```



```
Browser starts rendering
↓
dynamic Cart resolves
↓
RSC/streamed result
↓
React inserts Cart
```

So **streaming doesn't mean sending random HTML chunks with no React meaning.**

The server and React coordinate through the rendering/RSC machinery.

14. One HUGE misconception to avoid

Don't say:

❌ "Server Components send HTML and Client Components send JavaScript."

That's too simplistic.

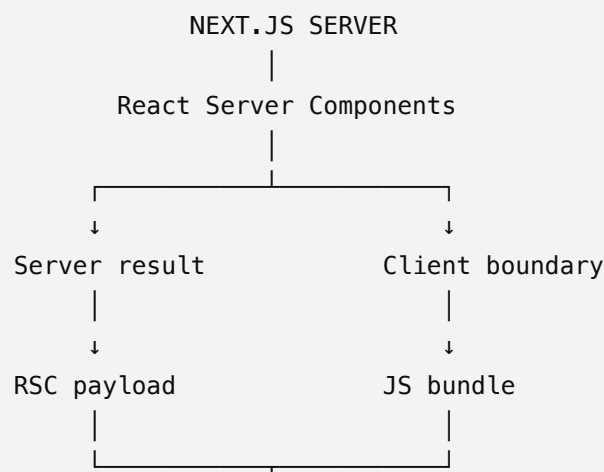
A better statement:

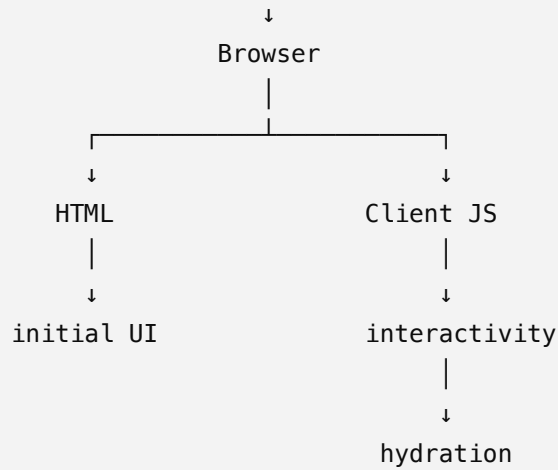
"Server Components are rendered on the server and their result is represented in the RSC payload; the initial response can also contain HTML for the browser to display. Client Components require JavaScript to run in the browser for interactivity."

That's much more accurate.

15. The complete architecture 🧠

Here's the diagram I want you to keep:





And for navigation:

```
<Link>
  ↓
Client Router
  ↓
request RSC payload
  ↓
Server
  ↓
RSC response
  ↓
React reconciles
  ↓
updated UI
```



Interview-ready answer

If they ask:

"What is the difference between HTML, RSC payload, and JavaScript in Next.js?"

Say:

"HTML provides the initial document markup that the browser can display. The RSC payload represents the rendered React Server Component tree and allows React and Next.js to update that tree, especially during client-side navigation. JavaScript is sent to the browser for Client Components that require interactivity, event handlers, and browser-side logic."

That's a very solid answer. 🔥

Next lesson

Now we should connect this to **hydration vs reconciliation vs client-side navigation**, because there's one subtle question left:

If Server Components don't ship their JavaScript to the browser, how does React know what the Server Component tree looks like during navigation?

That leads directly into **RSC payload + Flight + reconciliation**, which is probably the last major piece of this architecture puzzle. 🧩



wait before model full route cache used to contain the rendered output route but after migrating to vcache component it stores according rendered component or data so when it fetches to browser initially server component is coming with html+RSC payload and rsc payload later gates cached in browser cache right??

Yes — you're very close, Kanna. 👍 But there are **two important corrections**.

1. Old model vs Cache Components

Old App Router caching model

You can mentally picture:

```
Route /products/123
  ↓
Full Route Cache
  ↓
rendered route output
  ↓
HTML + RSC payload
```



So the **Full Route Cache** was **route-level**: the rendered result of the route could be stored and reused.

Modern Cache Components model

With Cache Components, don't think:

Full Route Cache



gone



component cache



Instead, the important mental model is:

"use cache"



cache a specific server computation/component result



For example:

getProduct(123)



cached result

Recommendations(123)



cached result

Header()



cached result



while:

Cart(currentUser)



dynamic



So yes, **the caching granularity becomes much more component/computation-oriented**, rather than your old mental model of "the entire route's rendered output is the cache unit."

2. But your second part needs correction

You said:

"server component is coming with HTML + RSC payload and RSC payload later gets cached in browser cache"

The first part:  broadly yes

For an **initial request**, Next.js can send:

```
Server
  ↓
HTML
+
RSC payload/data
+
Client Component JS where needed
  ↓
Browser
```



The HTML allows the browser to display the initial UI.

The RSC information is used by React/Next.js to understand the Server Component tree.

The second part: ⚠ not quite

Don't say:

```
RSC payload
  ↓
Browser Cache
```



as a general rule.

The important browser-side cache you've already learned is:

```
Router Cache
```



and it stores **RSC payloads / rendered Server Component segments** for client-side navigation.

So think:

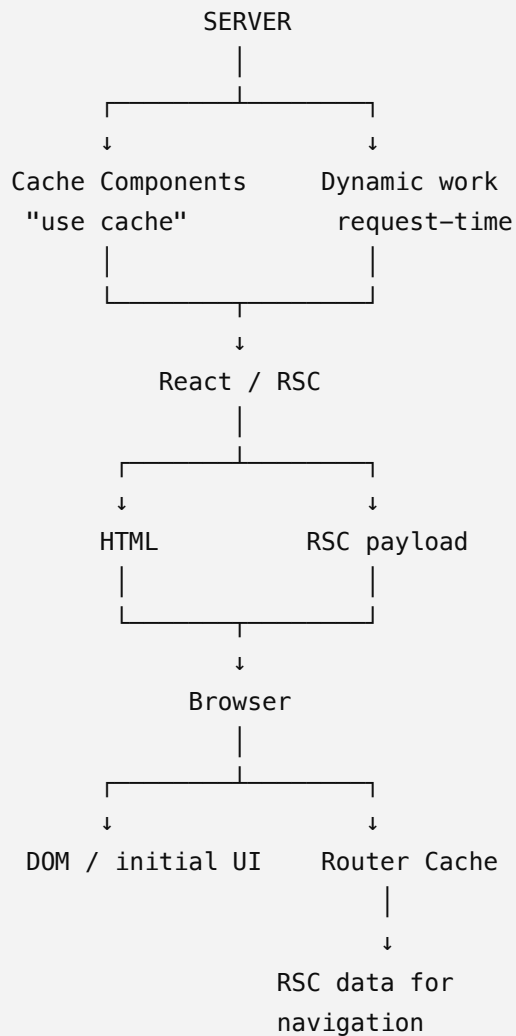
```
SERVER
  |
  | RSC payload
  ↓
BROWSER
  |
  ↓
Router Cache
```



That's different from the browser's ordinary HTTP cache.

3. Put all the layers together

This is the mental model you want:



And separately:



That's **not** the same thing as Router Cache.

4. One more subtle correction

Don't think:

"Cache Components stores either rendered components OR data."

Better:

`"use cache"` caches the result of a server-side computation.

That computation might be:

```
</> TypeScript
async function getProduct(id) {
  "use cache";
  return db.product.findUnique(...);
}
```

→ cached **data result**

or:

```
</> TypeScript
async function Product({ id }) {
  "use cache";
  ...
  return <ProductUI />;
}
```

→ cached **server-rendered component result**

So the precise mental model is:

```
"use cache"
  ↓
cache result of this server computation
```

not:

```
"use cache"
  ↓
put this component/data into some generic browser cache
```

🔥 Your understanding, corrected

What you said:

Old Full Route Cache = rendered route output
Modern Cache Components = more granular cached server computations/components

Initial request = HTML + RSC payload

RSC can then be retained in the browser's Router Cache for navigation

✅ That's the mental model.

Just remember this distinction:

"use cache"



SERVER-SIDE CACHE

Router Cache



BROWSER-SIDE CACHE



RSC payload / route segments



They are two completely different cache layers. 🧠

And this is exactly why `router.refresh()` doesn't simply mean "clear the browser cache" — it asks Next.js for a fresh Server Component result and then updates the current UI.



Yes — this is the **exact confusion to clear up**. 😊 Once you understand this, Server Components become much less magical.

1. "If Server Component code isn't in the browser, how does it show on screen?"

Because the browser doesn't need the Server Component code to display its result.

Think about a restaurant 🍕 :

```
Chef (server)
  ↓
uses recipe + ingredients
  ↓
prepares pizza
  ↓
waiter delivers pizza
  ↓
Customer sees pizza
```



The customer doesn't need the **recipe** or the **kitchen** to see the pizza.

Same idea:

```
SERVER
Server Component code
  ↓
executes
  ↓
produces result
  ↓
HTML / RSC payload
  ↓
BROWSER
  ↓
screen
```



2. Concrete example

You write:

```
</> TypeScript

// app/products/page.tsx

export default async function ProductsPage() {
  const products = await db.product.findMany();

  return (
    <main>
      <h1>Products</h1>

      {products.map(product => (
        <div key={product.id}>
          {product.name}
        </div>
      ))}
    </main>
  );
}
```



```
    </main>
  );
}
```

This is a **Server Component**.

The browser does NOT need:

- ✗ ProductsPage.tsx
- ✗ db.product.findMany()
- ✗ database credentials
- ✗ database connection



The server executes:

```
ProductsPage()  
  ↓  
DB query  
  ↓  
React renders result
```



Suppose the result is:

```
Products:  
iPhone  
MacBook  
AirPods
```



The browser can receive/render markup representing that result:

 HTML



```
<main>  
  <h1>Products</h1>  
  <div>iPhone</div>  
  <div>MacBook</div>  
  <div>AirPods</div>  
</main>
```

So the browser says:

"Cool, I have markup. I'll paint this on the screen."

It doesn't need to know **how the server produced it**.

3. Then what is RSC doing?

Here's the important distinction:

Server Component CODE

↓

runs on server

↓

Server Component RESULT

↓

RSC payload

↓

browser React



The browser receives the **result/representation**, not the original server implementation.

That's why:

✗ Browser:
"Let me execute ProductsPage.tsx"

vs

✓ Browser:
"I received the result of ProductsPage."



4. Can you SEE Router Cache in your browser? 🙄

Yes — but there's an important caveat.

You won't normally open DevTools and see:

Router Cache

/products/123

/products/456



as a nice readable cache panel.

Next.js doesn't expose Router Cache as a simple Chrome DevTools storage bucket like:

Application

├ Local Storage

├ Session Storage

├ IndexedDB

└ Router Cache ← ✗ not normally here



Router Cache is an **internal client-side cache maintained by the Next.js App Router**, not ordinary `localStorage`.

5. But you CAN inspect evidence of it 🔍

This is actually a great experiment for you.

Open your Next.js app in Chrome.

Then:

Chrome DevTools



Network



Navigate using:

</> TypeScript

```
<Link href="/products/123">
```



rather than:

HTML

```
<a href="/products/123">
```



You can inspect the network requests and see requests associated with the App Router/RSC navigation.

Depending on your Next.js version and dev/prod mode, you'll see RSC-related request/response information.

You may encounter headers such as:

RSC

Next-Router-State-Tree

Next-Router-Prefetch



and responses associated with the RSC protocol.

The exact presentation varies between development/production and Next.js versions.

6. Here's a really good experiment

Make two pages:

```
/products/123  
/products/456
```



and navigate:

```
123  
↓  
456  
↓  
123
```



using:

```
</> TypeScript  
  
<Link href="/products/123">
```



Watch:

```
DevTools  
→ Network
```



You'll be able to observe the App Router making RSC/navigation requests.

Then try:

```
/products/123
```



→

```
/products/456
```



→

```
/products/123
```



The Router Cache can allow previously received route information to be reused according to its current caching behavior.

7. Can you visually see the cached Server Component?

This needs a careful distinction.

✗ You generally won't see:

Next.js Server Cache

Product component
cached result

inside Chrome DevTools.

Why?

Because that cache is **on the server**, not in your browser.

Remember:

"use cache"

↓

SERVER-SIDE CACHE

while:

Router Cache

↓

CLIENT-SIDE

They're different.

8. So where can you inspect each one?

Thing	Where it lives	Can you inspect it?
Server Component source	Server/build	Not as browser JS
"use cache" result	Server-side cache	Not directly through normal browser DevTools
RSC response	Network	✔ Yes
HTML	Network → document response	✔ Yes

Thing	Where it lives	Can you inspect it?
Client JS	Network/Sources	✅ Yes
Router Cache	Next.js client runtime	⚠️ Not as a normal DevTools storage panel
localStorage	Browser	✅ Yes
sessionStorage	Browser	✅ Yes
HTTP/browser cache	Browser DevTools	✅ Partially inspectable

9. And here's a SUPER useful experiment for you

Open:

Chrome DevTools
→ Network
→ click the document request
→ Response



You'll be able to inspect the initial HTML response.

Then navigate with `<Link>` and inspect the subsequent requests.

You'll start seeing the difference between:

INITIAL LOAD

HTML
+
RSC information
+
Client JS



and:

CLIENT NAVIGATION

RSC-related response
+
existing browser document



That distinction is really important for App Router interviews.

10. One more thing: don't confuse "showing" with "executing"

This is the key sentence:

A browser can display the output of code without possessing or executing the code that generated that output.

For example:

Server Component:

```
<h1>{product.name}</h1>
  ↓
server knows product.name = "iPhone"
  ↓
result
  ↓
<h1>iPhone</h1>
  ↓
browser displays it
```

The browser doesn't need:

`</>` TypeScript

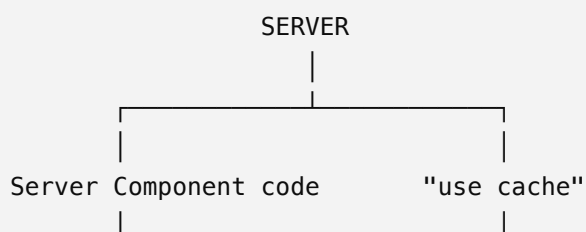
```
<h1>{product.name}</h1>
```

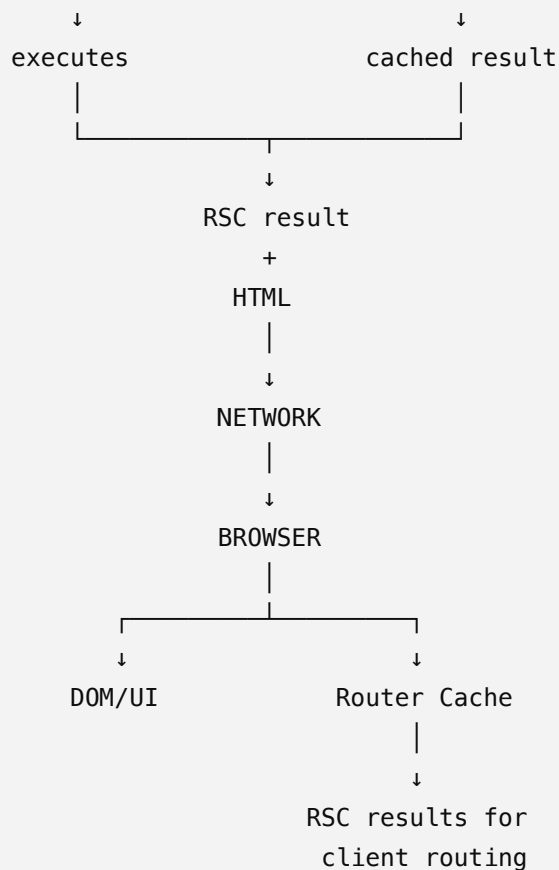
to be able to display:

iPhone

Final picture

Keep these two worlds separate:





🔥 The one thing to remember:

Server Component code stays on the server. Its *result* crosses the network.

And yes, **you can absolutely open DevTools and inspect the HTML and RSC network traffic**. You just generally can't open a little "Router Cache" folder and browse it like localStorage.

If you want, we can next do a **hands-on DevTools experiment** where I give you a tiny Next.js page and tell you **exactly what to click in Chrome DevTools and what you should expect to see**. That'll make this stick for real. 🙄



i willdo it after i build nextjs app ..next

Absolutely 🤪 We'll do the DevTools experiment **after you build the Next.js app** so you can actually see the HTML, RSC requests, navigation, and caching instead of just imagining them.

Next → Lesson 6.8 🚀

We'll move into:

Hydration vs Reconciliation vs RSC Updates

We'll answer:

Server Component renders on server

↓

HTML reaches browser

↓

Client Component JS arrives

↓

What exactly gets hydrated?

↓

What does React reconcile?

↓

What happens during router.refresh()?

↓

What happens during <Link> navigation?

And we'll use one small example throughout so the differences are crystal clear. 🧠

how any more?

Haha 😂 you're asking **how many lessons are left in the whole Next.js roadmap**, right?

You completed **Lesson 4**, and we're currently around **Lesson 6.7** in the deeper caching/RSC section.

 Remaining roadmap

Lesson	Topic	Status
6.8	Hydration vs Reconciliation vs RSC Updates	➡️ SOON Next
6.9	Full request lifecycle — putting everything together	🕒
7	Next.js Performance	🕒
8	Authentication & Authorization	🕒

Lesson	Topic	Status
9	Middleware / Proxy & request interception	🕒
10	Route Handlers & APIs	🕒
11	Error Handling & Loading UX	🕒
12	Forms, Server Actions & Mutations — interview deep dive	🕒
13	SEO, Metadata & OG Images	🕒
14	Images, Fonts & Scripts	🕒
15	Deployment, Vercel, CDN & Edge	🕒
16	Testing Next.js apps	🕒
17	Security & production best practices	🕒
18	Next.js System Design	🕒
19	Interview questions + debugging scenarios	🕒
20	Build the Web Performance Analyzer 🚀	🕒

So roughly 14 major lessons/sections remain.

But don't worry — several of those are much smaller than the caching/RSC section we've just gone through. 😂

And 6.8 → 6.9 is basically the final deep architectural piece. After that we'll move into practical/interview-focused Next.js topics.

Your journey so far

- Lesson 1 → Server vs Client Components

Lesson 2 → App Router & Routing

Lesson 3 → Navigation & Routing details

Lesson 4 → Data Fetching

Lesson 5 → Mutations & Server Actions

Lesson 6 → Caching + Rendering architecture
 - └ Cache Components
 - └ use cache
 - └ cacheLife
- ✓

✓

✓

✓

✓

🔄

✓

✓

✓



└─ cacheTag	✓
└─ PPR	✓
└─ RSC	✓
└─ 6.8 Hydration/Reconciliation	→ SOON

So we're **well past the fundamentals now.** 🔥

And after the roadmap, we'll use all of this in your **Web Performance Analyzer project**, which is where the concepts should really become second nature.



ChatGPT can make mistakes. Check important info.